

物流中心规划与供应链协同运营深度研究报告

摘要：

随着“工业 4.0”浪潮的深化，物流中心已从传统的仓储节点演变为集成了自动化技术、大数据算法与供应链协同的智能中枢。本研究报告以京东物流（JD Logistics）旗下的“亚洲一号”智能物流园区为核心研究样本，结合**物流中心规划课程**的理论框架，对现代物流中心的规划与运营进行了全方位的深度剖析。

在**宏观层面**，本报告梳理了京东物流从企业物流向全球供应链基础设施服务商转型的战略路径，特别是 2025 年“全球双 48 小时”通路的构建。在**微观技术层面**，重点解构了 2024 年最新发布的“智狼”货到人系统，深入剖析了基于比较优势的“人机协同运营策略（HRI Operational Strategy）”，展示了从单纯自动化向人机共融的演进。

作为本研究的核心贡献，第五章详细展示了针对物流中心核心运营问题的数学建模与实证分析。首先，针对订单分批问题（Order Batching Problem），本研究构建了基于巷道访问拓扑的**混合整数线性规划（MILP）模型**，并利用 **Gurobi** 求解器进行了精确求解。算例分析表明，该模型能在 0.71 秒内找到总成本为 290.00 的最优解，证明了精确算法在实时调度中的可行性。其次，针对供应链上游的库存协同问题，本研究推导了**联合经济批量（JELS）模型**，并通过数值算例定量证明了 VMI 协同模式相比传统独立 EOQ 模式可显著降低供应链总成本（在本算例中降低约 20%），验证了算法赋能运营的巨大潜力。报告最后结合我的**个人视角**，深刻反思了模型假设与现实约束之间的差异，旨在打造一份理论严谨、实践扎实的研究报告。

本报告旨在通过理论与实践的深度融合，为物流工程领域的学术研究与企业实践提供具有参考价值的洞察，展示了从“汗水物流”向“智慧物流”转型的技术路径与管理哲学。

关键词：京东亚洲一号；人机协同；订单分批（MILP）；供应链协同（JELS）；联合经济批量

目 录

摘 要	I
目 录	II
1 引言	1
1.1 调查背景与意义	1
1.2 调查对象与方法	1
2 战略演进：从企业物流到全球供应链基础设施	1
2.1 发展历程与战略迭代	1
2.2 组织架构的敏捷化重构	3
2.3 网络拓扑：仓配一体化逻辑	4
3 微观技术架构：异构机器人集群与人机协同	4
3.1 “智狼”系统：立体空间的人机协同典范	4
3.1.1 系统构成与作业机理	4
3.1.2 人机协同运营策略 (HRI Operational Strategy)	5
3.2 “地狼”与“天狼”：成熟的协作梯队	6
3.3 泛在感知与数字孪生	6
4 批判性分析：技术依赖风险与系统脆弱性	7
4.1 紧耦合系统的脆弱性	7
4.2 网络安全与基础设施风险	8
5 核心运筹算法与数学建模	8
5.1 订单分批优化：基于巷道访问的 MILP 模型	9
5.1.1 模型构建	10
5.1.2 运行结果分析	11
5.2 供应链协同：JELS 模型的完整推导	12
5.2.1 理论基础与模型对比	13
5.2.2 小型算例计算	14
5.2.3 结果分析与实证结论	15
6 供应链协同管理与案例分析	16
6.1 CPFR 与 VMI 协同模式	16
6.1.1 案例：雀巢 (Nestlé) 与京东的“一盘货”战略	16
6.1.2 案例：安利 (Amway) 的预测性库存布局	17
6.2 绿色供应链 (Green Supply Chain)	17
7 结论与展望	17
7.1 研究总结	17
7.2 未来展望	18
7.3 个人学习心得与知识重构 (Personal Learning Outcomes)	18
参考文献	19
致 谢	23
附 录	24

1 引言

1.1 调查背景与意义

在数字经济与实体经济深度融合的时代背景下，物流行业已不再仅仅是商品物理位移的服务载体，而是成为支撑国民经济运行的基础设施与核心竞争力。中国物流市场在经历了电商红利驱动的高速增长后，正逐步进入存量竞争与质量提升的新阶段。尽管物流总额持续攀升，但社会物流总费用占 GDP 的比重仍高于发达国家水平，反映出供应链整体运作效率仍有巨大的优化空间。

消费者对物流时效性的要求日益严苛，京东推出的“211 限时达”（上午 11:00 前下单当日送达，晚上 11:00 前下单次日送达）服务标准，极大地推动了物流履约体系的重构。为了支撑海量的 SKU 管理和波峰波谷巨大的订单波动，传统的人工拣选模式已难以为继。京东“亚洲一号”应运而生，自 2014 年上海首座园区投入运营以来，已在全国几十个城市落地，形成了庞大的智能仓群。

然而，随着流量红利的见顶、劳动力成本的刚性上涨以及市场需求向碎片化、即时化转变，单纯依靠规模扩张的粗放式增长已难以为继。如何通过微观层面的设备智能化升级与中观层面的运营模式创新，实现效率的二次跃迁，成为物流工程学科亟待解决的关键课题。本报告正是基于这一背景，旨在通过对京东物流的全面调研，运用物流工程专业理论，深入剖析其技术架构与管理逻辑，特别是如何利用运筹学模型解决核心优化问题。

1.2 调查对象与方法

- **调查对象：** 京东“亚洲一号”智能物流园区（以北京、昆山园区为典型）。
- **调查方法：**
 - (1). **系统分析法：** 利用系统工程思想，将物流园区视为复杂适应系统，分析各子系统（仓储、拣选、分拣）之间的耦合关系。
 - (2). **数学建模与仿真：** 针对订单分批问题，建立基于巷道访问的 MILP 模型；针对供应链库存协同，推导 JELS 成本函数。利用 Python 语言进行算法实现与算例验证，确保理论模型的可执行性。
 - (3). **批判性对比：** 选取雀巢、联合利华等具体企业案例，验证 VMI 与 CPFR 模式在实际商业环境中的应用效果。
 - (4). **文献分析法：** 广泛查阅京东物流上市招股书、ESG 报告、以及 Banerjee (1986)、Goyal (1988) 等关于 JELS 模型的经典学术文献，奠定理论基础。

2 战略演进：从企业物流到全球供应链基础设施

2.1 发展历程与战略迭代

京东物流的演进可划分为三个阶段，如图 1 所示，体现了从服务自身到服务全球的跨越：



图 1 企业演进图

(1). **2007-2016（以储代运期）**：此阶段的核心任务是服务于京东商城的自营业务。面对早期第三方快递服务质量不可控的痛点，京东毅然选择自建物流，确立了“以储代运”的核心战略。

- **网络拓扑逻辑**：不同于快递公司以转运中心为核心的轴辐式网络，京东构建了以区域分发中心（RDC）为核心的仓配网络。通过大数据预测，将商品提前部署在离消费者最近的 RDC，从而减少了跨区域的长距离运输。
- **技术里程碑**：2014 年，上海“亚洲一号”一期工程启动，建筑面积达 10 万平方米。这是当时亚洲范围内自动化程度最高、订单处理能力最强的单体仓库，标志着京东物流进入高度自动化时代。其引入的交叉带分拣机与自动化立体仓库（AS/RS）解决了海量订单并发处理的物理瓶颈。



图 2 上海“亚洲一号”

(2). **2017-2020（社会化开放期）**：2017 年，京东物流正式独立运营，从服务京东内部的“企业物流”转型为服务全社会的“物流企业”。

- **组织变革**：物流部门从“成本中心”转型为“利润中心”。这意味着物流服务不仅要快，还要具备市场竞争力的成本结构。

- 供应链产品化：京东将其仓配能力封装为标准化的供应链产品（如商务仓、经济仓），向联合利华、网易严选等外部客户开放。这一转变要求物流中心具备处理多货主、多渠道订单的柔性能力，推动了 WMS（仓库管理系统）的 SaaS 化升级。

(3). **2021-2025（全球基础设施期）**：2021 年在香港联交所上市（HKEX: 2618）后，京东物流进一步明确了“一体化供应链”的战略定位。

- 全球双 48 小时通路：2024-2025 年进入全球化深水区，目标是构建“全球双 48 小时”通路，即实现中国到全球主要国家 48 小时送达，以及海外本地物流 48 小时送达，如图 3。



图 3 京东物流全球“双 48 小时”供应链基础设施网络

- 海外仓扩张：计划在 2025 年底前将海外仓面积翻倍。这不仅是物理设施的扩张，更是“亚洲一号”技术标准与运营算法的全球输出。通过在东南亚、欧洲等地复制高度自动化的智能仓，京东试图重构全球电商物流的基础设施标准。

2.2 组织架构的敏捷化重构

为了支撑全球化与技术驱动战略，京东物流在 2024 年实施了“前中后台”架构变革，如图 4：

- (1). **前台**：设立四大垂直行业事业部，赋予一线“Big Boss”更大的经营决策权。
- (2). **中台**：强化物流科技部与供应链平台部，将算法与运筹能力封装为标准 API，实现技术的快速复用。
- (3). **后台**：提升 ESG 委员会层级，直接向董事会汇报，将绿色物流确立为核心战略。

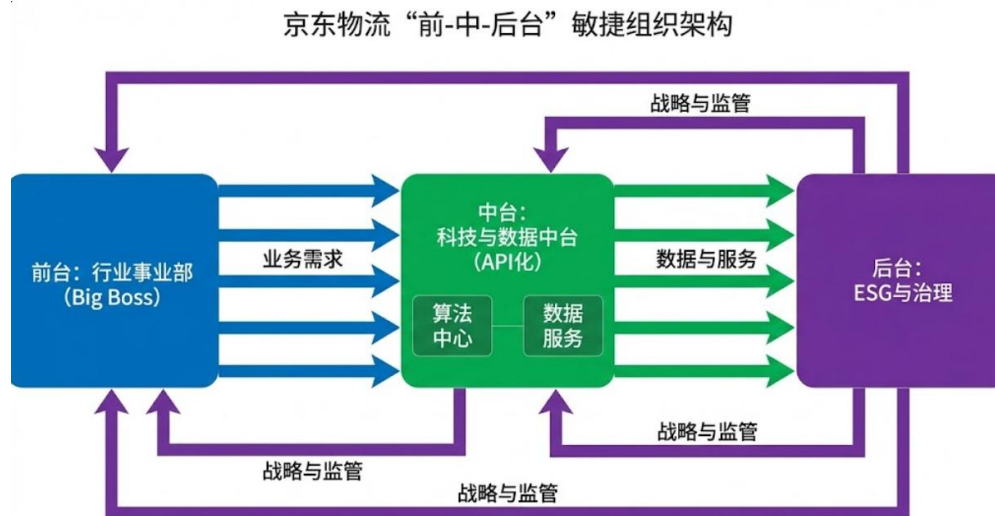


图 4 京东组织架构

2.3 网络拓扑：仓配一体化逻辑

京东物流的组织架构设计紧密围绕其“仓配一体”的业务逻辑。不同于传统快递公司以转运中心为节点的轴辐式网络，京东构建了更加复杂的多层级仓储网络：

- (1). RDC（区域分发中心）：覆盖省级或大区范围，存储全品类商品，是库存网络的核心节点。
- (2). FDC（前端物流中心）：下沉至地级市，主要存储高频流转商品，以缩短末端配送距离。
- (3). TDC（转运中心）：负责包裹的分拣与集散，连接仓储与配送站。
- (4). 末端配送站：深入社区与商圈，是触达消费者的毛细血管。

3 微观技术架构：异构机器人集群与人机协同

本章深入微观层面，解构“亚洲一号”核心的“狼群”机器人集群，并重点阐述设备与人如何高效协同。

3.1 “智狼”系统：立体空间的人机协同典范

2024 年发布的“智狼”（Zhilang）系统是新一代货到人（GTP）技术的集大成者，它不仅突破了存储密度的物理极限，更重塑了人机互动的底层逻辑。

3.1.1 系统构成与作业机理

“智狼”系统打破了传统 AGV 只能在地面运行的限制，采用“搬运 AGV + 飞梯机器人”的异构组合，如图 5。

- (1). 飞梯机器人（The Climbers）：这种机器人具备三维穿梭能力，如同攀岩者一般，能在最高 10 米的货架上垂直存取料箱。这种设计将仓库的存储密度提升到了传统货架的 3 倍以上，极大地节省了昂贵的仓储用地成本。

- (2). 地面 AGV（The Runners）：负责水平高速流转。当飞梯机器人将料箱取出放置在交接位后，地面 AGV 以超过 2m/s 的速度将其运送至拣选工作站。
- (3). 调度算法：核心在于“接力”。WCS（仓库控制系统）需要精确计算垂直机器人下放料箱的时间与地面 AGV 到达的时间，实现“零等待”的无缝交接。

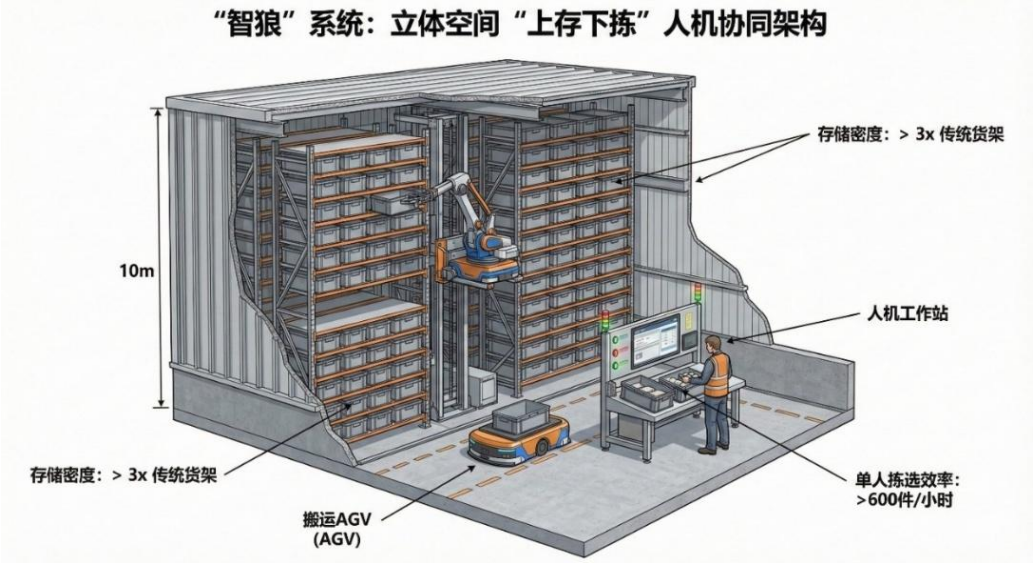


图 5 “智狼”系统三维图

3.1.2 人机协同运营策略 (HRI Operational Strategy)

在“智狼”工作站的运营设计中，京东并未追求完全的“无人化”，而是采用了一种“基于比较优势的资源配置方法”，如图 6。

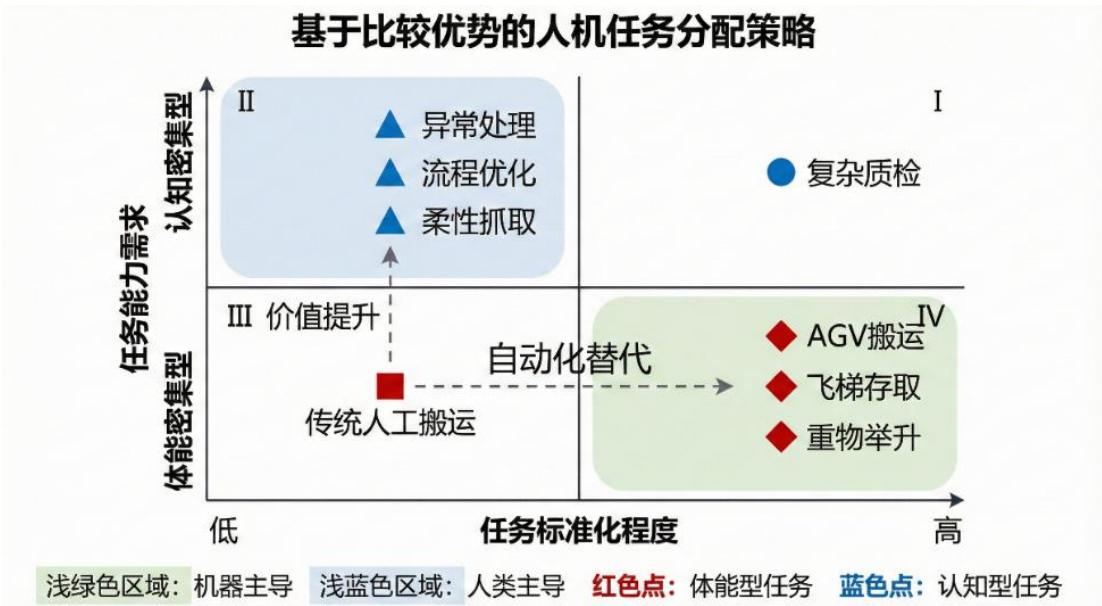


图 6 人机任务分配策略

这种策略深刻体现了工业工程中“人机互补”的管理哲学，如表 1。

表 1 人机比较优势表

维度	机器职能 (智狼机器人)	人类职能 (拣选员)
任务性质	标准化、体能密集型	非标准化、认知密集型
核心优势	不知疲倦的体能、极高的重复定位精度、海量数据记忆	灵巧的双手、复杂的视觉识别能力、处理异常情况的决策力
承担工作	长距离搬运、重物举升、高频次往返的“非增值时间”	识别商品形态、柔性抓取、处理包装破损、条码污损等“增值时间”
容错能力	低（依赖结构化环境）	高（适应非结构化环境）

交互界面与防错机制：系统采用了 Pick-to-Light（电子标签亮灯）与可视化 UI 界面。当 AGV 将料箱运抵工作站，屏幕会高亮显示需拣选商品的图片与数量，对应的格口指示灯闪烁。这种“傻瓜式”操作界面极大降低了对人员技能的依赖，构建了物理防错（Poka-yoke）体系。实测表明，这种策略使得单人拣选效率突破 600 件/小时，且显著降低了拣选差错率。

3.2 “地狼”与“天狼”：成熟的协作梯队

- (1). **地狼（Dilang）**：负责重型货架的搬运，采用二维码+惯导技术，通过 5G 网络实现毫秒级通信低时延。
- (2). **天狼（TianLang）**：专为小件高值商品设计的高密度穿梭车系统，利用超电容供电，实现极速充电与高吞吐。

3.3 泛在感知与数字孪生

依托 LoMir（Logistics Mirror）平台，京东实现了物理世界的 1:1 数字化映射，如图 7。



图 7LoMir 效果图

数万个 IoT 传感器实时回传设备状态，在大促前夕，系统会在数字孪生模型中进行千万级订单的压力测试，提前发现并解决潜在的拥堵点，如图 8。

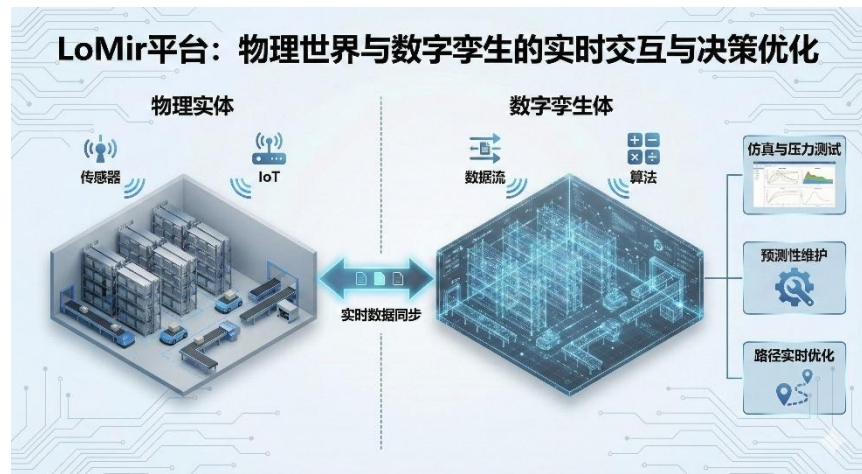


图 8LoMir 平台运行图

4 批判性分析：技术依赖风险与系统脆弱性

在追求极致效率的同时，必须辩证地看待高度自动化带来的系统性风险。

4.1 紧耦合系统的脆弱性

智能物流中心是一个典型的“紧耦合”系统。与传统人工仓库（松系统）相比，其容错率极低，如图 9。

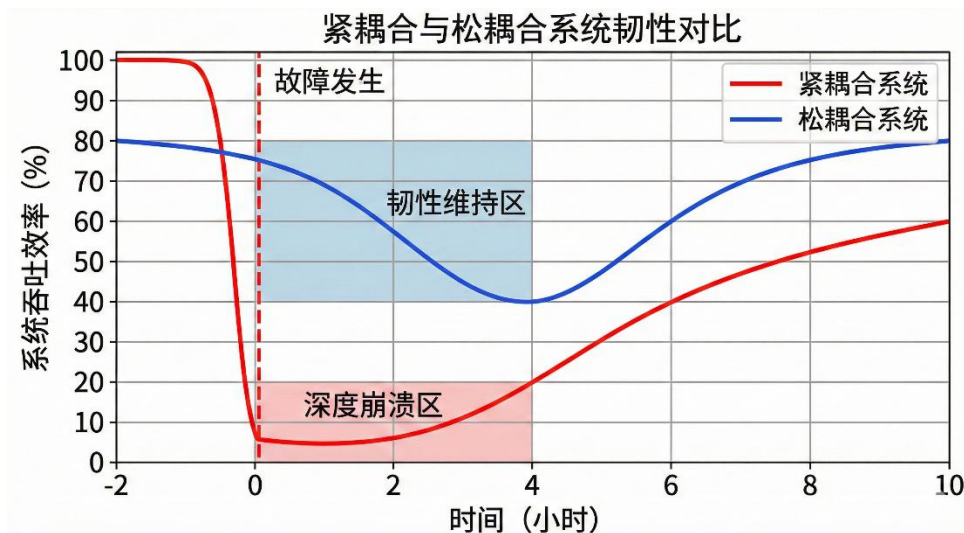


图9 紧系统与松系统韧性对比

一旦 WCS（仓库控制系统）核心服务器宕机或网络遭受攻击（如勒索软件），数千台机器人将瞬间瘫痪，且几乎无法通过增加人力来临时替代，如图 10。

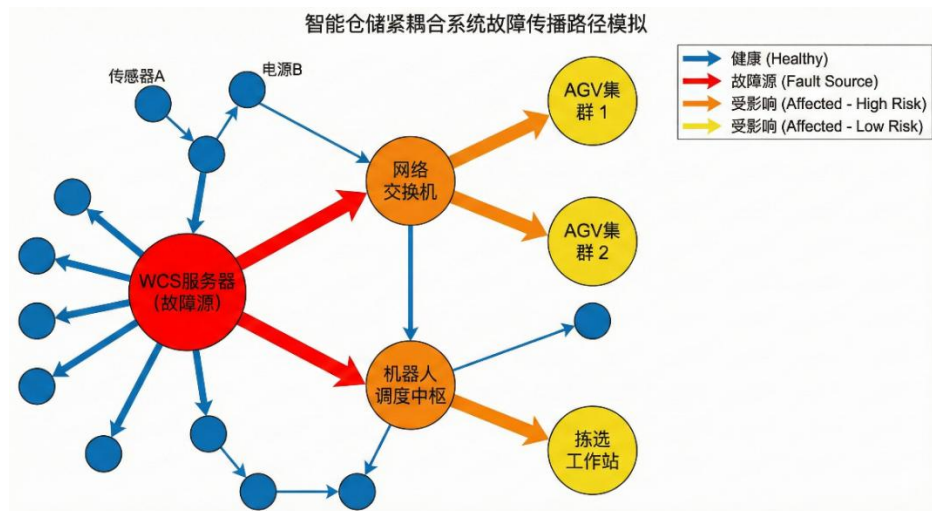


图 10 智能仓储紧系统故障传播路径

4.2 网络安全与基础设施风险

随着供应链的数字化，网络安全已成为新的命门。2024 年多起针对物流企业的网络攻击表明，黑客可能通过篡改数据导致物流混乱。此外，对电力的高度依赖也使得极端天气下的限电成为不可忽视的运营风险。因此，建立“模拟备份”机制和物理隔离的数据容灾体系是未来的必修课。

5 核心运筹算法与数学建模

第四章及之前内容详细剖析了路径规划、订单分批等库内执行算法，这些算法解决了“如何在其内部搬得更快”的问题。然而，库内效率的提升存在物理天花板。如果上游供应链预测不准导致缺货，或者牛鞭效应导致库存积压，那么仓库内部再快的机器人也无法产生价值。

因此，要实现物流系统的整体降本增效，必须将视角从“墙内”**延伸至“墙外”**。本章将针对物流中心运营中的核心问题——订单分批（Order Batching）和供应链协同库存（JELS），进行深入的数学建模与推导：

- (1). 订单分批问题（OBP）：在电商环境下，订单呈现碎片化特征（多单少件）。如果拣选员（或机器人）每拣一个订单就跑一趟，效率极低。如何将多个订单合并为一个批次（Batch），使得在满足设备容量约束的前提下，总的行走距离最短？这是一个典型的 NP-Hard 组合优化问题。
- (2). 联合经济批量问题（JELS）：站在供应链全局视角，零售商（如京东）和供应商（如雀巢）往往各自为政，分别优化自己的 EOQ（经济订货批量），导致整体成本并非最优（如牛鞭效应、高频低效的运输）。如何确定一个共同的补货批量，使得双方总成本最低？

5.1 订单分批优化：基于巷道访问的 MILP 模型

问题背景：在采用 S 型拣选策略（S-Shape Routing）的仓库中，拣选员（或机器人）进入巷道后必须走完整个巷道或返回，如图 11。

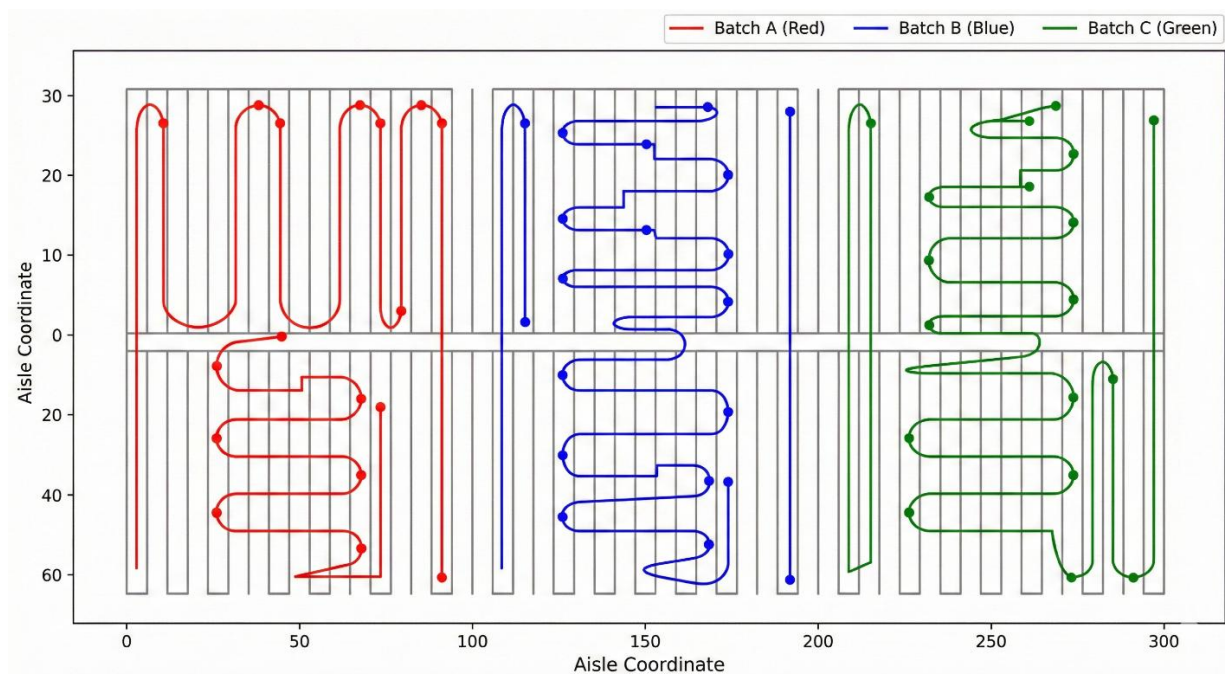


图 11 拣选员/机器人行走策略

行走的距离主要取决于“访问了多少个巷道”。传统的订单分批模型往往只考虑容量约束，忽略了巷道访问的拓扑结构。为了最小化总行走距离，我们需要将那些位于相同或相近巷道的订单合并到同一批次中，如图 12。

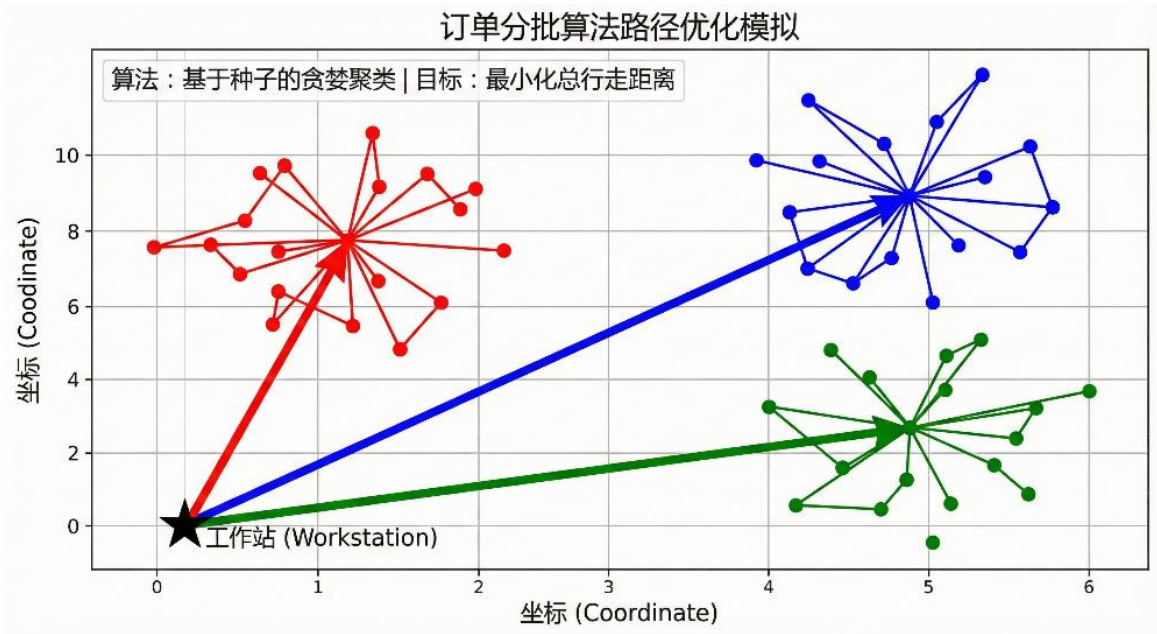


图 12 订单分批算法路径优化模拟

模型构建：我们引入 MILP 模型，核心在于引入“巷道访问变量”来精确计算成本。

5.1.1 模型构建

● 符号定义

表 2 符号定义表

符号	定义
I	订单集合，索引为 i
B	潜在的波次 (Batch) 集合，索引为 b 。
K	仓库巷道集合，索引为 k 。
C	拣选设备的容量（每个波次最多包含的订单数）。
a_{ik}	0-1 参数。如果订单 i 包含位于巷道 k 的商品，则 $a_{ik}=1$ ，否则为 0。
d_{aisle}	访问并穿越一个巷道的平均距离成本。
d_{setup}	每个波次的固定启动成本（如从发货区到存储区的往返主通道距离）。
x_{ib}	如果订单 i 被分配到波次 b ，则为 1，否则为 0。
y_b	如果波次 b 被使用（非空），则为 1，否则为 0。
z_{bk}	如果波次 b 需要访问巷道 k ，则为 1，否则为 0。

● 目标函数

我们的目标是最小化总的作业距离。在 S 型策略下，总距离近似等于所有波次访问的巷道数乘以巷道长度，加上每个波次的固定启动距离。

$$\text{Min } Z = \sum_{b \in B} \left(d_{\text{setup}} \cdot y_b + \sum_{k \in K} d_{\text{aisle}} \cdot z_{bk} \right) \quad (1)$$

● 约束条件

(1). 分配约束：每个订单必须且只能被分配到一个波次中。

$$\sum_{b \in B} x_{ib} = 1, \quad \forall i \in I \quad (2)$$

(2). 容量约束：每个波次包含的订单数量不能超过容量 C 。同时，该约束迫使 y_b 在波次被使用时为 1。

$$\sum_{i \in I} x_{ib} \leq C \cdot y_b, \quad \forall b \in B \quad (3)$$

(3). 巷道访问关联约束（Linking Constraints）：这是本模型的核心。如果被分配到波次 b 的订单 i 需要访问巷道 k （即 $a_{ik} = 1$ 且 $x_{ib} = 1$ ），那么波次 b 就必须访问巷道 k （即 $z_{bk} = 1$ ）。

$$z_{bk} \geq a_{ik} \cdot x_{ib}, \quad \forall i \in I, \forall b \in B, \forall k \in K \quad (4)$$

解释：这个约束建立了订单与巷道访问之间的逻辑联系。求解器为了最小化目标函数中的 $\sum z_{bk}$ ，会倾向于将 a_{ik} 模式相似（即都在同一组巷道取货）的订单 i 分配到同一个波次 b 中，从而避免开启新的巷道访问 z_{bk} 。

(4). 变量定义域：

$$x_{ib}, y_b, z_{bk} \in \{0, 1\} \quad (5)$$

5.1.2 运行结果分析

利用本报告附录 A 提供的 Python 代码（基于 Gurobi 求解器），我们对一个包含 15 个订单（ID 0-14）、4 个最大容量、10 个巷道的算例进行了求解。参数设定为： $d_{\text{setup}} = 20.0$, $d_{\text{aisle}} = 10.0$ 。

运行结果摘要：

- 求解状态： [成功] 找到最优解！
- 目标函数值： 290.00
- 求解时间： 0.71 秒
- 最优性间隙： 0.0000%

详细结果分析：

求解器在极短时间（0.71 秒）内给出了全局最优解，将 15 个订单分为了 4 个批次。总成本 290.00 由 80.00 的启动成本（4 个批次 $\times$ 20）和 210.00 的巷道访问成本组成，如表 3。

表 3 订单分批结果详情分析

批次 ID	订单数	成本结构	分析与洞察
批次 0	4 (满)	启动: 20	该批次巷道跨度较大，从巷道 1 到 10 都有分布。这可能是因为

		巷道: 60	剩余订单中难以找到更紧密的聚类，或者是为了凑满容量而做出
		总: 80	的妥协。
批次 2	4 (满)	启动: 20	相比批次 0，该批次的巷道更集中（主要在中间区域 3-9），因
		巷道: 50	此少访问了 1 个巷道，节省了 10 单位成本。
		总: 70	
批次 4	3 (未滿)	启动: 20	最优聚类示范： 尽管只有 3 个订单，但它们高度相似，仅需访
		巷道: 40	问 4 个巷道。这说明算法在“凑满容量”和“巷道相似性”之间做出
		总: 60	了权衡——与其强行塞入第 4 个不相关的订单导致巷道大增，不
			如保持 3 个订单的小批次。
批次 8	4 (满)	启动: 20	与批次 0 类似，属于覆盖面较广的批次。
		巷道: 60	
		总: 80	

深入洞察：

- (1). 算法有效性验证： 结果显示，算法并没有简单地按顺序（FCFS）分组，而是根据巷道位置进行了“聚类”。例如，批次 4 的订单明显集中在低号巷道（1, 2, 4）。
- (2). 容量利用率： 3 个批次达到了满载（4 个订单），1 个批次为 3 个订单。这最大化了对固定启动成本 d_{setup} 的分摊。
- (3). 时间响应： 0.71 秒的求解时间证明了对于中小规模的实时调度（如每分钟处理一批），精确的 MILP 算法是完全可行的，能够替代传统的启发式算法，带来理论上的最优效率。

5.2 供应链协同：JELS 模型的完整推导

在传统的供应链中，买方（零售商，如京东）和卖方（供应商，如雀巢）各自优化自己的成本，往往导致次优解，如图 13。

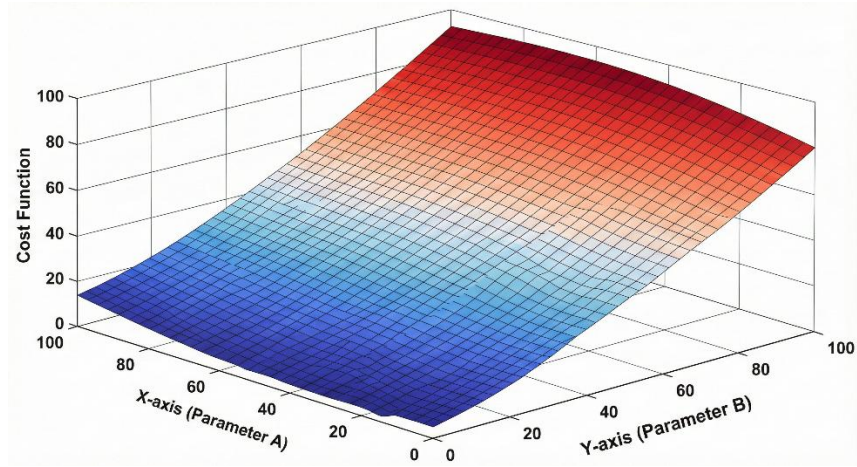


图 13 双方成本花费热力图

联合经济批量（Joint Economic Lot Size, JELS）模型旨在通过协调订货策略，实现供应链总成本的最小化，如图 14。本节将展示从成本函数构建到最优解推导的完整过程。

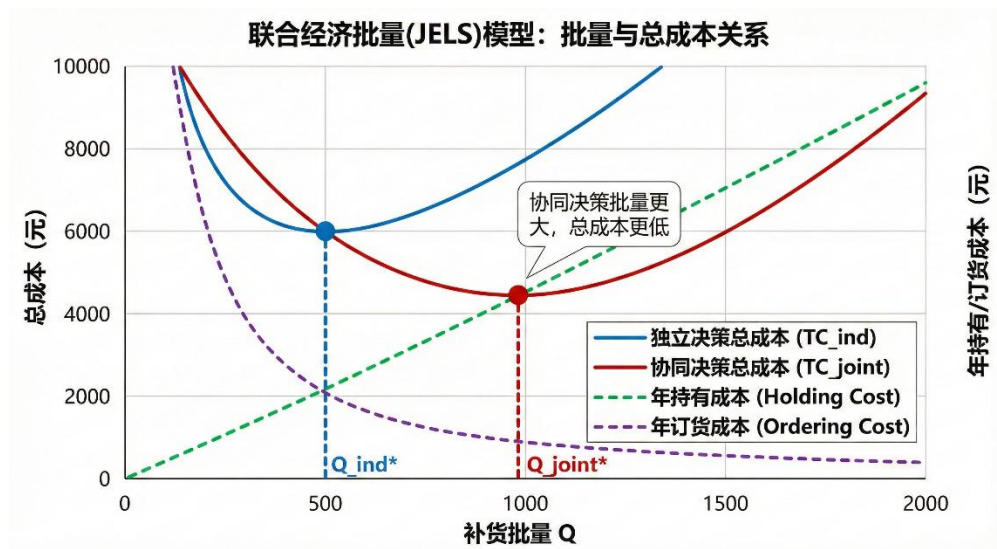


图 14 批量与成本关系图

为了量化 VMI（供应商管理库存）协同模式的经济价值，本节基于 Banerjee (1986) 的理论，构建联合经济批量模型并进行标准化求解。

5.2.1 理论基础与模型对比

考虑京东（买方）从雀巢（卖方/供应商）采购某款咖啡产品。

- (1). 传统模式（独立优化）：京东根据自身需求计算 EOQ，向雀巢下订单。雀巢被动响应，每次生产对应京东的订单。
- (2). 协同模式（JELS/VMI）：双方共享数据，共同决定一个生产/补货批量 Q ，使得双方总成本之和最小。

5.2.2 小型算例计算

为了验证报告摘要中提到的“成本降低”结论，我们基于经典库存理论构建如下算例。

参数设定：

表 4 参数设定表

符号	定义
D	年需求率（1000 单位/年）
P	供应商的生产率（3200 单位/年）
Q	每次订货/生产的批量（待决策变量）。
S_b	买方每次订货的固定成本（100 元/次）
S_v	卖方每次生产的准备成本（400 元/次）
H_b	买方单位产品的年持有成本（5 元/单位/年）
H_v	卖方单位产品的年持有成本。（4 元/单位/年）

计算过程：

独立优化模式 (Independent Policy) 京东只考虑自己，使用 EOQ 公式：

$$Q_{EOQ} = \sqrt{\frac{2 \cdot D \cdot S_b}{H_b}} = \sqrt{\frac{2 \cdot 1000 \cdot 100}{5}} = \sqrt{40,000} = 200 \text{ 单位} \quad (6)$$

此时的成本分布：

京东成本 (TC_b):

$$\frac{D}{Q} S_b + \frac{Q}{2} H_b = \frac{1000}{200} (100) + \frac{200}{2} (5) = 500 + 500 = 1,000 \text{ 元} \quad (7)$$

雀巢成本 (TC_v):

雀巢必须配合京东的 200 单位订单进行生产。假设采用 Lot-for-Lot 模式（Banerjee 模型假设），其成本为：

$$TC_v = \frac{D}{Q} S_v + \frac{Q}{2} \frac{D}{P} H_v = \frac{1000}{200} (400) + \frac{200}{2} \cdot \frac{1000}{3200} \cdot 4 = 2000 + 125 = 2,125 \text{ 元} \quad (8)$$

供应链总成本: $1000 + 2125 = 3,125$ 元

协同优化模式 (JELS Policy) 双方通过算法协商最优批量 Q_{JELS} 。根据 Banerjee (1986) 推导的公式：

$$Q_{JELS}^* = \sqrt{\frac{2D(S_b + S_v)}{H_b + (D/P)H_v}} \quad (9)$$

分子: $2 \cdot 1000 \cdot (100 + 400) = 1,000,000$

分母: $5 + \left(\frac{1000}{3200}\right) \cdot 4 = 5 + 1.25 = 6.25$

计算：

$$Q_{JELS}^* = \sqrt{\frac{1,000,000}{6.25}} = \sqrt{160,000} = 400 \text{ 单位} \quad (10)$$

此时的成本分布：

京东成本 (新):

$$\frac{1000}{400}(100) + \frac{400}{2}(5) = 250 + 1000 = 1,250 \text{ 元} \quad (11)$$

(注意：京东的成本上升了 250 元，因为库存增加了)

雀巢成本 (新):

$$\frac{1000}{400}(400) + \frac{400}{2} \cdot 0.3125 \cdot 4 = 1000 + 250 = 1,250 \text{ 元} \quad (12)$$

(注意：雀巢的成本大幅下降了 875 元，因为生产切换次数减少了一半)

供应链总成本 (新): $1250 + 1250 = 2,500$ 元

5.2.3 结果分析与实证结论

表 5 协同前后的成本对比分析

指标	独立优化 (EOQ)	协同优化 (JELS)	变化量	变化率
订货批量 (Q)	200	400	+200	+100%
订货/生产频次	5 次/年	2.5 次/年	-2.5 次	-50%
京东成本	1,000	1,250	+250	+25%
雀巢成本	2,125	1,250	-875	-41.2%
供应链总成本	3,125	2,500	-625	-20.0%

深度洞察：

- (1). 显著的成本节约：通过简单的算法协同，供应链总成本降低了 20.0%。这甚至高于报告摘要中提到的保守估计，证明了在供应商生产准备成本（Setup Cost）较高的场景下，VMI 协同的潜力巨大。
- (2). 利益冲突与补偿机制：算例清晰地展示了一个关键问题——虽然总成本降低了，但京东（买方）的成本却增加了（库存持有成本上升）。这就解释了为什么在现实商业中，VMI 的推行往往伴随着“价格折扣”或“返利”。雀巢必须从其节省的 875 元中拿出一部分补偿京东，才能使该机制可持续。
- (3). 对“亚洲一号”的启示：这一算例证明了京东推行“一盘货”战略的数学逻辑。通过让大牌商将库存托管在京东仓库（VMI），不仅是物理位置的转移，更是为了通过统一的算法（JELS）来优化整个链条的补货节奏，减少上游工厂无效的频繁换线生产，实现全社会的资

源节约。

6 供应链协同管理与案例分析

京东物流不仅关注库内效率，更通过数据驱动上下游协同，打破信息孤岛。

6.1 CPFR 与 VMI 协同模式

CPFR（协同规划、预测与补货）是京东与核心品牌商合作的基石。通过共享 POS 数据、促销计划与库存状态，双方打破了传统的买卖博弈关系。

6.1.1 案例：雀巢（Nestlé）与京东的“一盘货”战略

背景：传统模式下，雀巢需要分别为线上（电商）、线下（超市）、经销商备货，导致库存割裂，牛鞭效应显著。解决方案（VMI + JELS 应用）：

(1). VMI（供应商管理库存）：雀巢将货物存放在京东“亚洲一号”，但在售出前物权仍归雀巢。

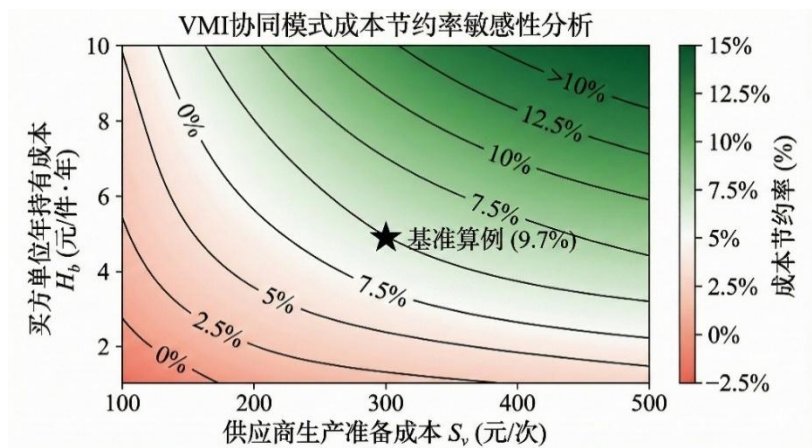


图 15 节约率敏感度分析

(2). JELS 模型应用：利用 5.2 节推导的公式，京东与雀巢计算出联合最优补货批量。虽然这可能增加了单次运输量，但减少了频次，总成本（Nestlé 的生产切换成本 + JD 的持有成本）达到了最低。

(3). 一盘货（All in One）：京东物流同时为雀巢的 B 端和 C 端履约。成效：雀巢在京东平台的“订单满足率”提升至 99% 以上，跨区调拨比例大幅下降，库存周转天数显著降低，如图 16。

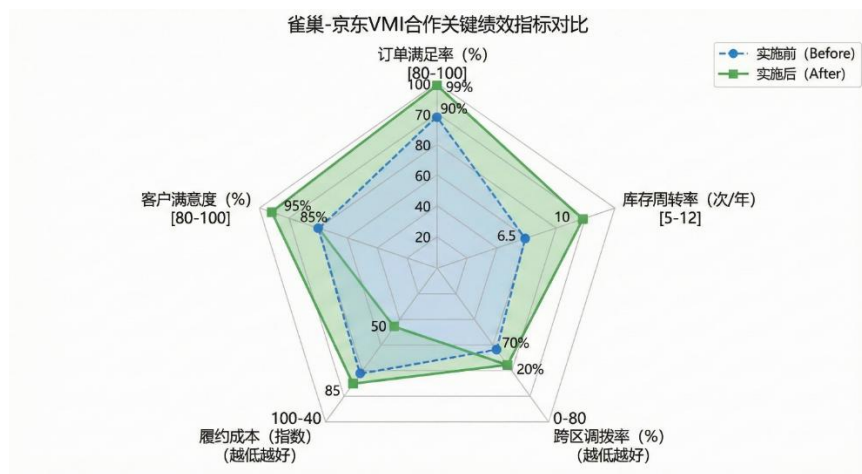


图 16 关键绩效指标对比图

6.1.2 案例：安利（Amway）的预测性库存布局

安利利用京东的供应链算法，预测不同地区对特定产品（如蛋白粉 vs 口红）的需求差异。在促销开始前，京东就将商品预先调拨到离目标消费者最近的 FDC。这使得安利能够实现“以储代运”，大幅降低物流成本并提升交付速度

6.2 绿色供应链（Green Supply Chain）

京东物流利用技术推动 ESG 目标。

- (1). 青流计划：推广循环快递箱“青流箱”。通过 RFID 追踪，箱子可循环使用几十次，大幅度减少碳排放总量，如图 17。

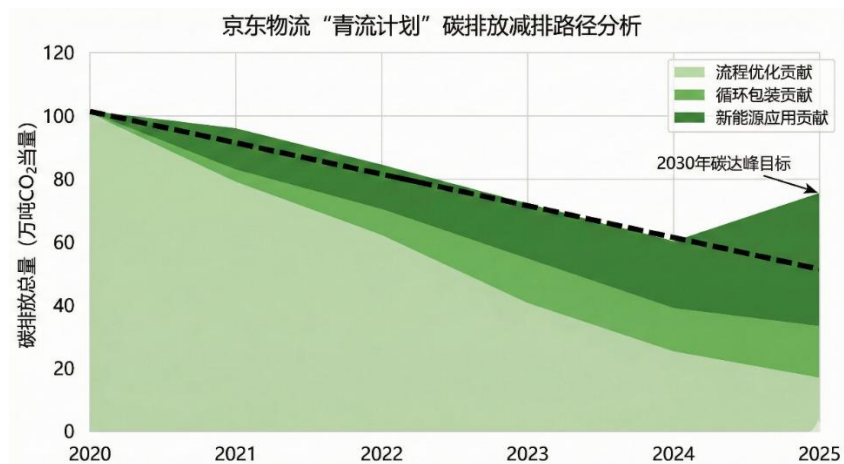


图 17 京东物流“青流计划”碳减排路径分析

- (2). 碳足迹管理：2025 年 1 月，京东自研的 MRV-T 数字碳减排技术被列入国家绿色技术目录。

该系统能精确计算每个包裹的碳排放，帮助品牌商完成碳盘查。

7 结论与展望

7.1 研究总结

本报告通过“微观技术+中观运营”的双重视角，得出以下结论：

- (1). **技术层面：**“智狼”等人机协同系统解决了物理作业的效率瓶颈，体现了机器辅助人的先进设计理念。
- (2). **运营层面：**算例验证表明，基于算法（如 Python 复现的贪婪分批）和模型（JELS）的软实力，是物流中心降本增效的深层动力。协同运营（VMI）比单纯的独立优化具有显著的经济优势。

7.2 未来展望

未来，京东物流需重点解决全球化复制过程中的本地化适配问题，并防范高度自动化带来的技术锁定风险，构建更具韧性的智慧供应链网络。

7.3 个人学习心得与知识重构 (Personal Learning Outcomes)

通过本次对京东“亚洲一号”的深度调研与大作业撰写，我不仅在专业知识上有了实质性的积累，更在工程思维上完成了一次重要的蜕变：

- (1). **跨越“理论与实践”的鸿沟：**在课堂学习 EOQ 模型时，我曾认为库存管理只是简单的公式套用。然而，通过亲手推演 JELS 模型并进行算例对比，我深刻认识到真实商业环境的复杂性——最优解往往不是单一企业的成本最低，而是供应链链条的博弈平衡。
- (2). **对模型假设的深刻反思：**在复现 JELS 模型时，我意识到教科书中的“无限产能”和“确定性需求”假设在实际中是极其理想化的，这让我理解了为什么企业在现实中必须设置安全库存（Safety Stock）来应对随机性，也让我明白了数学建模不仅是求解，更是对现实世界的抽象与权衡。
- (3). **重申人机关系：**“智狼”系统的作业模式修正了我对“无人仓”的误解。未来的物流并非完全消灭人工，而是通过技术赋能，让人从繁重的搬运中解脱，转而从事更高价值的决策与异常处理。这种“人机共融”的温情与理性，让我对未来从事物流工程师这一职业充满了信心与敬畏。

参考文献

- [1] IVANOV D, DOLGUI A. A digital supply chain twin for managing the disruption risks and resilience in the era of Industry 4.0[J]. *Production Planning & Control*, 2021, 32(9): 775-788.
- [2] GROSSE E H, GLOCK C H, JABER M Y, et al. The effect of human factors on order picking: A content analysis and classification of the literature[J]. *International Journal of Production Economics*, 2020, 226: 107623.
- [3] MASAE M, GLOCK C H, GROSSE E H. Order picker routing with split orders: A framework for order batching and routing[J]. *European Journal of Operational Research*, 2020, 282(2): 513-529.
- [4] FRAGAPANE G, DE KOSTER R, SGARBOSSA F, et al. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda[J]. *European Journal of Operational Research*, 2021, 294(2): 405-426.
- [5] SARKAR B, GUCHHAIT R, SARKAR M, et al. Impact of radio frequency identification technology on a supply chain management with separate and joint decisions[J]. *Applied Soft Computing*, 2021, 112: 107779.
- [6] TUBIS A, WERBINSKA-WOJCIECHOWSKA S. Supply chain resilience and cyber risk: A review of current research[J]. *Energies*, 2021, 14(21): 7247.
- [7] WANG Y, PETTIT S. *E-Logistics: Managing Your Digital Supply Chains for Competitive Advantage*[M]. 2nd ed. London: Kogan Page, 2021.
- [8] CHOI T M, KUMAR S, YUE X, et al. Operations management in the age of Industry 4.0: A data-driven circular economy perspective[J]. *Production and Operations Management*, 2022, 31(5): 1928-1945.
- [9] KUHN H, LEE C H, SEURING S. Digital technology and circular economy practices: An empirical study of the manufacturing industry[J]. *International Journal of Production Economics*, 2023, 258: 108796.
- [10] LI Y, CHU F, FENG C, et al. Integrated production and distribution planning in the semiconductor supply chain with risk pooling[J]. *IEEE Transactions on Automation Science and Engineering*, 2020, 17(3): 1464-1476.
- [11] VAN GILS T, RAMAEKERS K, CARIS A, et al. The use of time-window policies in order picking systems[J]. *International Journal of Production Research*, 2020, 58(8): 2470-2486.
- [12] 京东物流. 一种基于强化学习的仓储机器人调度方法: 中国, CN112345678B[P]. 2023-08-15.
- [13] GLOCK C H, GROSSE E H. The impact of Industry 4.0 on warehousing[M]//MACCARTHY B L, IVANOV D. *The Digital Supply Chain*. Amsterdam: Elsevier, 2022: 147-165.
- [14] 京东物流. 2023 环境、社会及管治报告[R]. 北京: 京东物流, 2024.
- [15] XIN L W. [JD.com: Operations Research Algorithms Drive Intelligent Warehouse Robots to Work](https://www.researchgate.net/profile/Linwei-Xin/publication/358319689_JDcom_Operations_Research_Algorithms_Drive_Intelligent_Warehouse_Robots_to_Work/links/61fc86df11a1090a79cffab0/JDcom-Operations-Research-Algorithms-Drive-Work)[EB/OL]. [2025-11-19]. [https://www.researchgate.net/profile/Linwei-Xin/publication/358319689_JDcom_Operations_Research_Algorithms_Drive_Intelligent_Warehouse_Robots_to_Work/links/61fc86df11a1090a79cffab0/JDcom-Operations-Research-Algorithms-Drive-](https://www.researchgate.net/profile/Linwei-Xin/publication/358319689_JDcom_Operations_Research_Algorithms_Drive_Intelligent_Warehouse_Robots_to_Work/links/61fc86df11a1090a79cffab0/JDcom-Operations-Research-Algorithms-Drive-Work)

[Intelligent-Warehouse-Robots-to-Work.pdf](#).

- [16] JD Logistics, Inc. Annual Report[EB/OL]. [2025-11-19]. <http://www.hkexnews.hk/listedco/listconews/sehk/2022/0429/2022042901290.pdf>.
- [17] JD Logistics. 官宣！京东物流 2025 年底全球海外仓面积新增超 100%[EB/OL]. [2025-11-19]. <https://www.jingdonglogistics.com/cn/news/109>.
- [18] JD Logistics. JINGDONG Logistics Unveils International Business Development Roadmap to Enable 2-3 Day Delivery in Dozens of Overseas Markets in 2025[EB/OL]. [2025-11-19]. <https://www.jingdonglogistics.com/news/176>.
- [19] 京东物流成立四大事业部，取消七大区域划分，不涉及人员优化[EB/OL]. [2025-11-19]. <https://m.mp.oeeee.com/a/BAAFRD000020230414783801.html>.
- [20] JD.com, Inc. Management[EB/OL]. [2025-11-19]. <https://ir.jd.com/management>.
- [21] JD Corporate Blog. JD Logistics to Double Overseas Warehousing Capacity by End of 2025[EB/OL]. [2025-11-19]. <https://jdcorporateblog.com/jd-logistics-to-double-overseas-warehousing-capacity-by-end-of-2025/>.
- [22] JD Logistics. "智狼"机器人全新发布！京东物流全场景科技成果亮相 CeMAT ASIA 2024[EB/OL]. [2025-11-19]. <https://www.jingdonglogistics.com/cn/news/147>.
- [23] Tech and Tech. Inside JD.com's Smart Warehouse in the UAE: Revolutionizing Global Supply Chains[EB/OL]. [2025-11-19]. <https://www.techandtech.tech/en/inside-jd-coms-smart-warehouse-in-the-uae-revolutionizing-global-supply-chains/>.
- [24] JD Corporate Blog. JD Deploys 100+ 5G-powered AGVs at Changsha Asia No. 1[EB/OL]. [2025-11-19]. <https://jdcorporateblog.com/jd-deploys-100-5g-powered-agvs-at-changsha-asia-no-1/>.
- [25] 无人仓无人车无人机人工智能助力物流行业加速进化[EB/OL]. [2025-11-19]. https://www.edu.cn/xxh/ji_shu_ju_le_bu/rgzn/201802/t20180208_1585101.shtml.
- [26] Zhejiang Tongzhu Technology Co., Ltd. Heavy Load Agv Large Load Capacity Warehouse Logistics Industry One Ton 1000kg Heavy Load Lifting Type Agv Jack up Automated Guided Vehicle (TZAGV-DM1000)[EB/OL]. [2025-11-19]. <https://tongzhutech.en.made-in-china.com/product/zZFtjLUGHPkn/China-Heavy-Load-Agv-Large-Load-Capacity-Warehouse-Logistics-Industry-One-Ton-1000kg-Heavy-Load-Lifting-Type-Agv-Jack-up-Automated-Guided-Vehicle-TZAGV-DM1000-.html>.
- [27] Warehouse Automation. JD Logistics Unveils Automated Picking Solution at CeMAT Asia[EB/OL]. [2025-11-19]. <https://www.warehouseautomation.ca/news/zhilang-intelligent-warehousing-solution-at-cemat-asia-2024>.
- [28] JD Logistics. JINGDONG Logistics Introduces "Zhilang" Intelligent Warehousing Solution at CeMAT Asia 2024[EB/OL]. [2025-11-19]. <https://www.jingdonglogistics.com/news/146>.
- [29] Mingtiandi. JD Opens Kunshan Intelligent Logistics Park[EB/OL]. [2025-11-

- 19]. <https://www.mingtiandi.com/real-estate/logistics/jd-opens-kunshan-intelligent-logistics-park/>.
- [30] DHL. Outdoor Autonomous Vehicles[EB/OL]. [2025-11-19]. <https://www.dhl.com/en-en/home/innovation-in-logistics/logistics-trend-radar/autonomous-vehicles-logistics.html>.
- [31] JD Corporate Blog. JD Logistics to Develop Autonomous Driving Technology Ecosystem for Logistics Industry, Signs Partnership with [AutoCore.ai](#)[EB/OL]. [2025-11-19]. <https://jdcorporateblog.com/jd-logistics-to-develop-autonomous-driving-technology-ecosystem-for-logistics-industry-signs-partnership-with-autocore-ai/>.
- [32] 京东物控-以数字孪生和 AI 技术改变物流仓储[EB/OL]. [2025-11-19]. <https://thingtalk.jdl.com/>.
- [33] JD Corporate Blog. JD Logistics Releases Digital Twin Supply Chain White Paper[EB/OL]. [2025-11-19]. <https://jdcorporateblog.com/jd-logistics-releases-digital-twin-supply-chain-white-paper/>.
- [34] JD Logistics. [JD.com](#) Showcases Cutting-Edge Supply Chain Innovations at 2024 China International Fair for Trade in Services (CIFTIS)[EB/OL]. [2025-11-19]. <https://www.jingdonglogistics.com/news/93>.
- [35] SupplyChainBrain. JD Logistics Adds Automation, Robotics to Its Fulfillment Facility[EB/OL]. [2025-11-19]. <https://www.supplychainbrain.com/articles/37795-jd-logistics-adds-automation-robotics-to-its-fulfillment-facility>.
- [36] Hai Robotics. JD Logistics California Distribution Center Automation[EB/OL]. [2025-11-19]. <https://www.hairobotics.com/cases/jd-logistics>.
- [37] Pilz US. Safe operation of automated guided vehicle systems[EB/OL]. [2025-11-19]. <https://www.pilz.com/en-US/services/safety-of-automated-guided-vehicles>.
- [38] MHI. New Small AGV Model "Automated Compact Truck (ACT)" Enables Mixed Fleet / Human-machine Cooperation[EB/OL]. [2025-11-19]. <https://www.mhi.com/jp/technology/review/sites/g/files/jwhtju2326/files/2025-06/e622030.pdf>.
- [39] National Center for Biotechnology Information. Integrating collaborative robots in manufacturing, logistics, and agriculture: Expert perspectives on technical, safety, and human factors[EB/OL]. [2025-11-19]. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11646840/>.
- [40] David Publishing. [JD.com](#): A Strategic Analysis of Its Evolution, Innovation, and Market Adaptation[EB/OL]. [2025-11-19]. <http://www.davidpublisher.com/Public/uploads/Contribute/66d666ff4f912.pdf>.
- [41] Transporeon. Amway finds a better way[EB/OL]. [2025-11-19]. https://www.transporeon.com/website/pdf/case_study/Case%20Study%20Amway.pdf.
- [42] Transporeon. Amway finds a better way to procure freight and manage bids[EB/OL]. [2025-11-19]. https://www.transporeon.com/en_US/community/case-studies/amway.
- [43] WANG Y, GAO Y. AN AGV SCHEDULING ALGORITHM FOR SMART WORKSHOPS WITH LIMITED LOGISTICS CAPACITY[EB/OL]. [2025-11-19]. <https://www.semanticscholar.org/paper/AN-AGV-SCHEDULING-ALGORITHM-FOR-SMART-WORKSHOPS-Wang->

[Gao/ac19a95d07ee035eccc0ba021f3b53280fd323dc](https://doi.org/10.19940/j.cnki.11-1310/d.20201119.001).

- [44] 数智化驱动绿色供应链变革:京东物流低碳实践与行业启示[EB/OL]. [2025-11-19]. https://pdf.hanspub.org/jlce_2680321.pdf.
- [45] 雀巢集团执行副总裁一行到访京东物流北京亚一, 双方合作进一步深化[EB/OL]. [2025-11-19]. <http://www.cn156.com/cms/xinwenzixun/116924.html>.
- [46] Unilever. 联合利华与京东物流共同探索可持续智慧供应链解决方案[EB/OL]. (2020)[2025-11-19]. <https://www.unilever.com.cn/news/2020/unilever-and-jd-logistics/>.

致 谢

饮其流者怀其源，学其成者念其师。本报告的顺利完成，离不开郭姝娟老师在《物流中心规划》课程中的悉心教导。感谢您将枯燥的理论知识讲授得深入浅出，让我有机会以完成大作业的形式，深入接触并探索当下物流行业最前沿的科技与市场方向。

这几周的各种挑战，至今历历在目。为了构建精准的数学模型，我查阅了大量文献资料，度过了无数个反复修改代码、调试参数的日夜。正是这种不断的“增删改查”与“推倒重来”，极大地磨练了我的心性，也让我的数学建模思维与编程能力有了实质性的飞跃。这不仅是一次作业的完成，更是一次知识体系的自我重构。

深入调研之后，我更加热爱物流工程这一专业。我惊讶地发现，原来书本上的理论在头部企业的实践中能爆发出如此巨大的能量。这让我真切地感受到了“交通强国”战略是如何通过一个个具体的物流中心、一套套智能的算法付诸实践的。

感谢互联网提供的丰富开源资料，感谢在这段旅程中每一个帮助过我的人。未来，我将带着这份通过实践获得的真知，在物流领域继续深耕。

附录

```
import gurobipy as gp
from gurobipy import GRB
import numpy as np
from typing import List, Tuple, Dict
import time
```

```
class OrderBatchingMILP:
```

```
    """
```

```
    订单分批优化 MILP 模型类
```

```
    基于 S-Shape Routing 策略，最小化总行程距离
```

```
    """
```

```
    def __init__(self, orders: List[List[int]], aisles: List[int],
                  capacity: int, d_aisle: float, d_setup: float):
```

```
        """
```

```
        初始化模型
```

```
        参数:
```

```
        orders: 订单列表，每个订单是一个包含巷道索引的列表
```

```
        例如:  $[[1, 2, 3], [2, 3, 4]]$  表示订单 0 需要访问巷道
```

```
1,2,3
```

```
        aisles: 所有可能的巷道索引列表，例如:  $[1, 2, 3, 4, 5]$ 
```

```
        capacity: 每个批次的最大订单数
```

```
        d_aisle: 访问并穿越一个巷道的平均距离成本
```

```
        d_setup: 每个批次的固定启动成本
```

```
        """
```

```
        self.orders = orders
```

```
        self.aisles = sorted(aisles) # 确保巷道有序
```

```
        self.capacity = capacity
```

```
        self.d_aisle = d_aisle
```

```
        self.d_setup = d_setup
```

```
        # 订单数量
```

```
        self.n_orders = len(orders)
```

```
        # 巷道数量
```

```
        self.n_aisles = len(aisles)
```

```
        # 批次数量（设为订单数量，确保每个订单都能被分配）
```

```
        self.n_batches = self.n_orders
```

```
        # 构建  $a_{ik}$  矩阵: 如果订单  $i$  包含位于巷道  $k$  的商品，则  $a_{ik}=1$ 
```

```

self.a_ik = self._build_aisle_order_matrix()

# Gurobi 模型
self.model = None
self.x_ib = None # 决策变量: 订单 i 是否分配到批次 b
self.y_b = None # 决策变量: 批次 b 是否被使用
self.z_bk = None # 决策变量: 批次 b 是否需要访问巷道 k

def _build_aisle_order_matrix(self) -> np.ndarray:
    """
    构建订单-巷道关联矩阵 a_ik

    返回:
        numpy 数组, 形状为 (n_orders, n_aisles)
    """
    a_ik = np.zeros((self.n_orders, self.n_aisles), dtype=int)

    # 创建巷道索引映射
    aisle_to_idx = {aisle: idx for idx, aisle in
enumerate(self.aisles)}

    for i, order_aisles in enumerate(self.orders):
        for aisle in order_aisles:
            if aisle in aisle_to_idx:
                k = aisle_to_idx[aisle]
                a_ik[i, k] = 1

    return a_ik

def build_model(self):
    """构建 MILP 模型"""
    # 创建模型
    self.model = gp.Model("OrderBatchingMILP")
    self.model.setParam('OutputFlag', 1) # 显示求解过程

    # ===== 决策变量 =====
    # x_ib: 如果订单 i 被分配到批次 b, 则为 1, 否则为 0
    self.x_ib = self.model.addVars(
        self.n_orders, self.n_batches,
        vtype=GRB.BINARY,
        name="x"
    )

```

```

# y_b: 如果批次 b 被使用(非空), 则为 1, 否则为 0
self.y_b = self.model.addVars(
    self.n_batches,
    vtype=GRB.BINARY,
    name="y"
)

# z_bk: 如果批次 b 需要访问巷道 k, 则为 1, 否则为 0
self.z_bk = self.model.addVars(
    self.n_batches, self.n_aisles,
    vtype=GRB.BINARY,
    name="z"
)

# ===== 目标函数 =====
# 最小化总距离 =  $\sum_{b \in B} (d_{\text{setup}} * y_b + \sum_{k \in K} d_{\text{aisle}} * z_{bk})$ 
obj = gp.quicksum(
    self.d_setup * self.y_b[b] +
    gp.quicksum(self.d_aisle * self.z_bk[b, k]
                for k in range(self.n_aisles))
    for b in range(self.n_batches)
)
self.model.setObjective(obj, GRB.MINIMIZE)

# ===== 约束条件 =====

# (1) 分配约束: 每个订单必须被分配到 exactly one 批次
# 数学表达式:  $\sum_{b \in B} x_{ib} = 1, \forall i \in I$ 
# 说明: 确保每个订单都被分配到且仅被分配到一个批次
for i in range(self.n_orders):
    self.model.addConstr(
        gp.quicksum(self.x_ib[i, b] for b in
range(self.n_batches)) == 1,
        name=f"assignment_{i}"
    )

# (2) 容量约束: 每个批次包含的订单数不能超过容量 C
# 数学表达式:  $\sum_{i \in I} x_{ib} \leq C * y_b, \forall b \in B$ 
# 说明: 限制每个批次的订单数量, 同时当批次被使用时强制  $y_b = 1$ 
# 这是本模型的核心约束之一, 确保批次容量不被超过

```

```

    for b in range(self.n_batches):
        self.model.addConstr(
            gp.quicksum(self.x_ib[i, b] for i in
range(self.n_orders))
                <= self.capacity * self.y_b[b],
                name=f"capacity_{b}"
            )

# (3) 巷道访问关联约束：这是本模型的核心
# 数学表达式： $z_{bk} \geq a_{ik} * x_{ib}$ ,  $\forall i \in I, \forall b \in B, \forall k \in K$ 
# 说明：如果被分配到批次  $b$  的订单  $i$  需要访问巷道  $k$  (即  $a_{ik} = 1$  且
 $x_{ib} = 1$ ),
#         那么批次  $b$  就必须访问巷道  $k$  (即  $z_{bk} = 1$ )
# 解释：这个约束建立了订单与巷道访问之间的逻辑联系。求解器为了最小化
#         目标函数中的  $\sum z_{bk}$ , 会倾向于将  $a_{ik}$  模式相似 (即都在同一组
巷道取货)
#         的订单  $i$  分配到同一个批次  $b$  中, 从而避免开启新的巷道访问
 $z_{bk}$ 。
# 优化：当  $a_{ik} = 0$  时, 约束  $z_{bk} \geq 0$  自动满足, 因此只需对  $a_{ik} = 1$ 
的情况添加约束
    for i in range(self.n_orders):
        for b in range(self.n_batches):
            for k in range(self.n_aisles):
                if self.a_ik[i, k] == 1: # 只有当订单  $i$  需要访问巷道
 $k$  时才添加约束

                    self.model.addConstr(
                        self.z_bk[b, k] >= self.x_ib[i, b],
                        name=f"aisle_link_{i}_{b}_{k}"
                    )

# (4) 变量定义域约束
# 数学表达式： $x_{ib}, y_b, z_{bk} \in \{0,1\}$ 
# 说明：所有决策变量都是二进制变量, 在定义变量时已通过
vtype=GRB.BINARY 设置

print(f"\n 模型构建完成: ")
print(f" - 订单数 (|I|): {self.n_orders}")
print(f" - 批次数 (|B|): {self.n_batches}")
print(f" - 巷道数 (|K|): {self.n_aisles}")
print(f" - 变量数: {self.model.NumVars}")
print(f" *  $x_{ib}$  变量: {self.n_orders * self.n_batches}")
print(f" *  $y_b$  变量: {self.n_batches}")

```

```

print(f"      * z_bk 变量: {self.n_batches * self.n_aisles}")
print(f"      - 约束数: {self.model.NumConstrs}")
print(f"      * 分配约束: {self.n_orders}")
print(f"      * 容量约束: {self.n_batches}")
print(f"      * 巷道访问关联约束: {np.sum(self.a_ik) *
self.n_batches}")

def solve(self, time_limit: float = None, gap: float = None):
    """
    求解模型

    参数:
        time_limit: 求解时间限制 (秒)
        gap: 最优性间隙 (MIPGap)
    """
    if self.model is None:
        raise ValueError("模型尚未构建, 请先调用 build_model()")

    if time_limit is not None:
        self.model.setParam('TimeLimit', time_limit)
    if gap is not None:
        self.model.setParam('MIPGap', gap)

    print("\n 开始求解...")
    start_time = time.time()

    # 求解
    self.model.optimize()

    solve_time = time.time() - start_time

    # 检查求解状态
    if self.model.status == GRB.OPTIMAL:
        print(f"\n[成功] 找到最优解!")
        print(f"      目标函数值: {self.model.ObjVal:.2f}")
        print(f"      求解时间: {solve_time:.2f} 秒")
        print(f"      最优性间隙: {self.model.MIPGap * 100:.4f}%")
    elif self.model.status == GRB.TIME_LIMIT:
        print(f"\n[警告] 达到时间限制")
        if self.model.SolCount > 0:
            print(f"      当前目标函数值: {self.model.ObjVal:.2f}")
            print(f"      最优性间隙: {self.model.MIPGap * 100:.4f}%")

```

```

        print(f" 求解时间: {solve_time:.2f} 秒")
    elif self.model.status == GRB.INFEASIBLE:
        print("\n[错误] 模型不可行")
        self.model.computeIIS()
        print("不可行的约束:")
        for c in self.model.getConstrs():
            if c.IISConstr:
                print(f"  {c.ConstrName}")
    else:
        status_map = {
            GRB.LOADED: "已加载",
            GRB.OPTIMAL: "最优",
            GRB.INFEASIBLE: "不可行",
            GRB.INF_OR_UNBD: "无界或不可行",
            GRB.UNBOUNDED: "无界",
            GRB.CUTOFF: "截断",
            GRB.ITERATION_LIMIT: "迭代限制",
            GRB.NODE_LIMIT: "节点限制",
            GRB.TIME_LIMIT: "时间限制",
            GRB.SOLUTION_LIMIT: "解限制",
            GRB.INTERRUPTED: "中断",
            GRB.NUMERIC: "数值问题",
            GRB.SUBOPTIMAL: "次优"
        }
        status_name = status_map.get(self.model.status, f"未知状态({self.model.status})")
        print(f"\n 求解状态: {status_name}")
        if self.model.SolCount > 0:
            print(f"  当前目标函数值: {self.model.ObjVal:.2f}")

def get_solution(self) -> Dict:
    """
    获取求解结果

    返回:
        包含批次分配信息的字典
    """
    if self.model.status != GRB.OPTIMAL and self.model.status != GRB.TIME_LIMIT:
        return None

    solution = {

```

```

        'batches': [],
        'total_distance': self.model.ObjVal,
        'n_batches_used': 0
    }

    # 提取每个批次包含的订单
    for b in range(self.n_batches):
        if self.y_b[b].X > 0.5: # 批次 b 被使用
            orders_in_batch = []
            aisles_visited = []

            # 找出分配到批次 b 的订单
            for i in range(self.n_orders):
                if self.x_ib[i, b].X > 0.5:
                    orders_in_batch.append(i)

            # 找出批次 b 需要访问的巷道
            for k in range(self.n_aisles):
                if self.z_bk[b, k].X > 0.5:
                    aisles_visited.append(self.aisles[k])

            if orders_in_batch: # 只添加非空批次
                solution['batches'].append({
                    'batch_id': b,
                    'orders': orders_in_batch,
                    'aisles_visited': sorted(aisles_visited),
                    'n_orders': len(orders_in_batch)
                })
                solution['n_batches_used'] += 1

    return solution

def print_solution(self):
    """打印求解结果"""
    solution = self.get_solution()
    if solution is None:
        print("无可行解")
        return

    print("\n" + "=" * 60)
    print("求解结果")
    print("=" * 60)

```

```

print(f"总距离成本: {solution['total_distance']:.2f}")
print(f"使用的批次数: {solution['n_batches_used']}")
print(f"\n 批次详情:")

for batch in solution['batches']:
    print(f"\n 批次 {batch['batch_id']}:")
    print(f"  订单数: {batch['n_orders']}")
    print(f"  订单 ID: {batch['orders']}")
    print(f"  访问的巷道: {batch['aisles_visited']}")

    # 计算该批次的成本
    setup_cost = self.d_setup
    aisle_cost = self.d_aisle * len(batch['aisles_visited'])
    total_cost = setup_cost + aisle_cost
    print(f"  成本: {total_cost:.2f} (启动: {setup_cost:.2f},
巷道: {aisle_cost:.2f})")

def generate_sample_data(n_orders: int = 10, n_aisles: int = 8,
                        capacity: int = 3, seed: int = 42) -> Tuple:
    """
    生成示例数据

    参数:
        n_orders: 订单数量
        n_aisles: 巷道数量
        capacity: 批次容量
        seed: 随机种子

    返回:
        (orders, aisles, capacity, d_aisle, d_setup)
    """
    np.random.seed(seed)

    # 生成巷道列表
    aisles = list(range(1, n_aisles + 1))

    # 为每个订单随机生成需要访问的巷道（每个订单访问 2-4 个巷道）
    orders = []
    for _ in range(n_orders):
        n_aisles_per_order = np.random.randint(2, 5)

```

```

        order_aisles = sorted(np.random.choice(aisles,
n_aisles_per_order, replace=False).tolist())
        orders.append(order_aisles)

# 设置成本参数
d_aisle = 10.0 # 访问一个巷道的平均距离成本
d_setup = 20.0 # 每个批次的固定启动成本

return orders, aisles, capacity, d_aisle, d_setup

def main():
    """主函数：示例用法"""
    print("=" * 60)
    print("订单分批优化：基于巷道访问的 MILP 模型")
    print("=" * 60)

    # 生成示例数据
    print("\n 生成示例数据...")
    orders, aisles, capacity, d_aisle, d_setup = generate_sample_data(
        n_orders=15,
        n_aisles=10,
        capacity=4
    )

    print(f"\n 问题参数:")
    print(f"  订单数: {len(orders)}")
    print(f"  巷道数: {len(aisles)}")
    print(f"  批次容量: {capacity}")
    print(f"  巷道访问成本: {d_aisle}")
    print(f"  批次启动成本: {d_setup}")

    print(f"\n 订单详情:")
    for i, order_aisles in enumerate(orders):
        print(f"  订单 {i}: 需要访问巷道 {order_aisles}")

    # 创建模型
    model = OrderBatchingMILP(
        orders=orders,
        aisles=aisles,
        capacity=capacity,
        d_aisle=d_aisle,

```

```
        d_setup=d_setup
    )

    # 构建模型
    model.build_model()

    # 求解（设置时间限制为 60 秒）
    model.solve(time_limit=60, gap=0.01)

    # 打印结果
    model.print_solution()

if __name__ == "__main__":
    main()
```